



Livello Trasporto

Parte 2

Internet, Web e Cloud
Claudia Canali

Principi del trasferimento dati affidabile

Trasferimento affidabile: principi generali



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Consideriamo i **principi del trasferimento dati affidabile** in un contesto generale

Problema: trasferimento dati affidabile su infrastruttura inaffidabile

TCP è affidabile → TCP gestisce un **trasferimento affidabile e ordinato** di uno stream di dati

- Quali **meccanismi** dobbiamo mettere in atto per **garantire questo servizio affidabile** considerando un “canale” sottostante che può causare la **perdita** o la **corruzione** dei pacchetti?
 - *Errori sui bit, perdita pacchetti, arrivo fuori ordine*
- *I meccanismi messi in atto dal TCP sono molteplici e complessi, andiamo per gradi....*

Protocolli incrementali



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Protocollo v1.0:** Trasferimento su canale completamente affidabile
 - caso ideale, nessun problema
 - *Il mittente spedisce il pacchetto e può assumere che venga correttamente ricevuto*
- **Protocollo v2.x:** Trasferimento su un canale con errore sui bit (no perdita pacchetti o consegna fuori ordine)
 - v2.0 - versione base con due tipi di ACK
 - v2.1 - risolve i problemi di duplicazione usando numeri di sequenza
 - v2.2 – semplifica evitando l'uso di due tipi di ACK
- **Protocollo v3.0:** Trasferimento su un canale con errore sui bit e perdita di pacchetti (*caso reale*)

Trasferimento affidabile: protocollo v2.0



- Consideriamo il trasferimento su un canale con **possibile errore** sui bit ma **senza perdita** di pacchetti e con **consegna in ordine**
- Il canale trasmissivo può solo modificare il **valore di uno o più bit**
- Possibile **modalità** per recuperare l'errore: **feedback esplicito dal destinatario** dopo il controllo di errore (*checksum*)
- **Utilizzo di notifiche (acknowledgement)**
 - Le notifiche possono essere **positive (ACK)** o **negative (NAK)**
 - **Acknowledgement (ACK)**: il destinatario comunica esplicitamente al mittente che il pacchetto ricevuto è OK
 - **Acknowledgement negativo (NAK)**: il destinatario comunica esplicitamente al mittente che il pacchetto ricevuto ha un errore
 - Il mittente **ritrasmette** il pacchetto se riceve un NAK
- In generale, i protocolli di trasferimento affidabili basati su ritrasmissioni sono detti **ARQ (Automatic Repeat Question)**

Trasferimento affidabile: protocollo v2.0



- Assumiamo per ora un **protocollo STOP-AND-WAIT**
 - Dopo l'invio di un pacchetto, il **mittente si mette in attesa** di una notifica (ACK o NAK)
 - Le notifiche potrebbero essere costituite anche da un solo bit (es. 1 per ACK e 0 per NAK)
 - Se riceve ACK, il mittente sa che il pacchetto è stato correttamente ricevuto e può spedire il successivo
 - Se riceve NAK, il mittente rispedisce l'ultimo pacchetto e si rimette in attesa di risposta
- ***Che problemi vedete?***

Trasferimento affidabile: protocollo v2.0



- **Problema:** anche i pacchetti ACK e NAK potrebbero essere corrotti
 - Potremmo introdurre dei bit di checksum alle notifiche
 - MA come **reagire** a un errore nei pacchetti ACK/NAK?
 - *A causa del danneggiamento, il mittente non è sicuro che il destinatario abbia ricevuto correttamente o meno il pacchetto*
 - In caso di notifica alterata, il mittente potrebbe rispedire il pacchetto
 - Questo però introduce la presenza di **pacchetti duplicati**
 - Il destinatario non ha modo di sapere se l'ultimo suo ACK o NAK sia stato ricevuto correttamente, quindi non può sapere se un pacchetto in arrivo contiene dati nuovi o ritrasmessi
 - Gestione impossibile di pacchetti duplicati

Trasferimento affidabile: protocollo v2.1



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Soluzione:** introduzione di un **numero di sequenza**
- **Protocollo v2.1:** trasferimento su un canale **con errore sui bit e gestione dei numeri di sequenza**
 - **Gestione numeri di sequenza**
 - il mittente aggiunge un numero di sequenza a ciascun pacchetto
 - il mittente **ritrasmette** il pacchetto se il messaggio di ACK/NAK è danneggiato (e quindi scartato)
 - Grazie al numero di sequenza, il destinatario può **scartare** (nel senso che non consegna al livello superiore) **i pacchetti duplicati**

Nell'ipotesi di un sistema di comunicazione STOP-AND-WAIT

→ **è sufficiente un bit (0,1) come numero di sequenza**

- Se il numero di sequenza è lo stesso → il mittente ha rispedito un pacchetto
- Se il numero di sequenza è diverso → il mittente ha spedito un nuovo pacchetto

Data l'ipotesi che il canale non perda pacchetti, non serve che i pacchetti ACK e NAK indichino il numero di sequenza del pacchetto di cui rappresentano la notifica

- ACK e NAK generati come risposta al pacchetto trasmesso più di recente

Necessità di uno 'stato' nel destinatario: il destinatario deve controllare se il pacchetto ricevuto è **dupplicato** (lo stato consente di verificare se il numero di sequenza del pacchetto è quello atteso o meno: es. attendo 1 e arriva 0 → duplicato)

Protocollo v2.2: possibile miglioramento



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Osservazione: nel protocollo v2.1, per ogni pacchetto ricevuto, il destinatario deve **inviare sempre un pacchetto di ACK o NAK al mittente**

Possibile miglioramento:

- Inviare **solo messaggi di ACK** per il più recente pacchetto ricevuto correttamente
- Il messaggio di ACK deve riportare il **numero di sequenza** del pacchetto ricevuto
- Il mittente controlla il numero di sequenza confermato dall'ACK ricevuto
- Il mittente che riceve due ACK per lo stesso pacchetto (**ACK duplicati**) sa che il destinatario non ha ricevuto correttamente il pacchetto successivo a quello confermato due volte e lo ritrasmette

Protocollo v2.2 → **Protocollo NAK-free**

Protocollo v2.2: possibile miglioramento



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Osservazione: nel protocollo v2.1, per ogni pacchetto ricevuto, il destinatario deve **inviare sempre un pacchetto di ACK o NAK al mittente**

Possibile miglioramento:

- **Inviare solo messaggi di ACK per il più recente pacchetto ricevuto correttamente**
- Il messaggio di ACK deve riportare il **numero di sequenza** del pacchetto ricevuto
- Il mittente controlla il numero di sequenza confermato dall'ACK ricevuto
- Il mittente che riceve due ACK per lo stesso pacchetto (**ACK duplicati**) sa che il destinatario non ha ricevuto correttamente il pacchetto successivo a quello confermato due volte e lo ritrasmette

Protocollo v2.2 → **Protocollo NAK-free**

Protocollo v2.2: riassumendo



DESTINATARIO

- Se il pacchetto **i** ricevuto è corretto (pacchetto integro), inviare un ACK con etichetta **i**
- Se il pacchetto **i** ricevuto non è corretto e il pacchetto **i-1** era corretto, inviare un ACK con etichetta **i-1**

MITTENTE

- Se il mittente riceve due ACK per il pacchetto **i-1**, può dedurre che il destinatario non ha ricevuto correttamente il pacchetto **i** e quindi deve agire come se fosse un (NAK, i) → **ritrasmettere il pacchetto i**

Protocollo v3.0: nuova ipotesi



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Assumiamo ora che il canale di trasmissione possa causare oltre ad **errori sui bit** anche **perdita di pacchetti** (*siano essi dati o ACK*)

Perdita: evento non raro su moderne reti di trasmissione

Come rilevare la perdita? E come reagire?

Come realizzare una trasmissione affidabile?

- *L'uso di checksum, numeri di sequenza dei pacchetti, ACK, ritrasmissioni sono utili, ma non sufficienti*
- *Che succede se il pacchetto si perde? Il mittente resta in attesa all'infinito*

SOLUZIONE → **COINVOLGERE IL “TEMPO”**

- Dopo aver spedito il pacchetto **i**, il **mittente** attende la conferma (ACK, **i**) per un **intervallo di tempo definito e “ragionevole”**
- Passato il tempo, se non ha ricevuto l'ACK, **ritrasmette** il pacchetto

NOTA: Assegnamo al mittente il compito di rilevare la perdita

Protocollo v3.0: trasferimento affidabile



NECESSITA'

1) Intervallo di **tempo ragionevole**

- Non facile da stimare, scelto in modo che la perdita sia probabile anche se non garantita
- Richiede l'uso di un **countdown timer**

***Sicuramente più di un RTT +
tempo di elaborazione da
parte del destinatario***

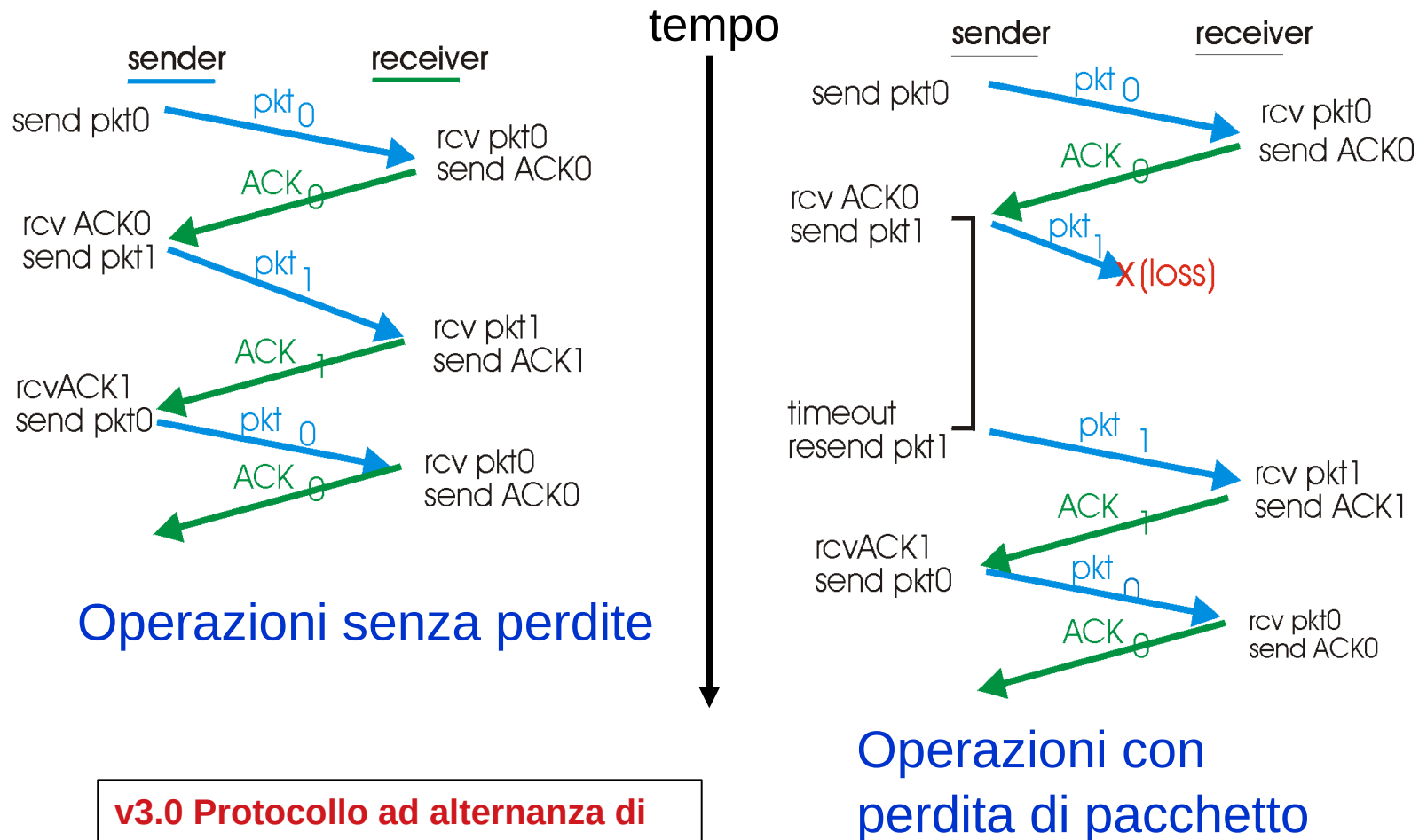
2) Se il pacchetto (messaggio o ACK) è in ritardo (ma non perso), il **pacchetto ritrasmesso risulterà duplicato**

- Questo caso è già risolto dall'uso dei **numeri di sequenza** (il destinatario quindi scarterà un eventuale pacchetto duplicato in arrivo)

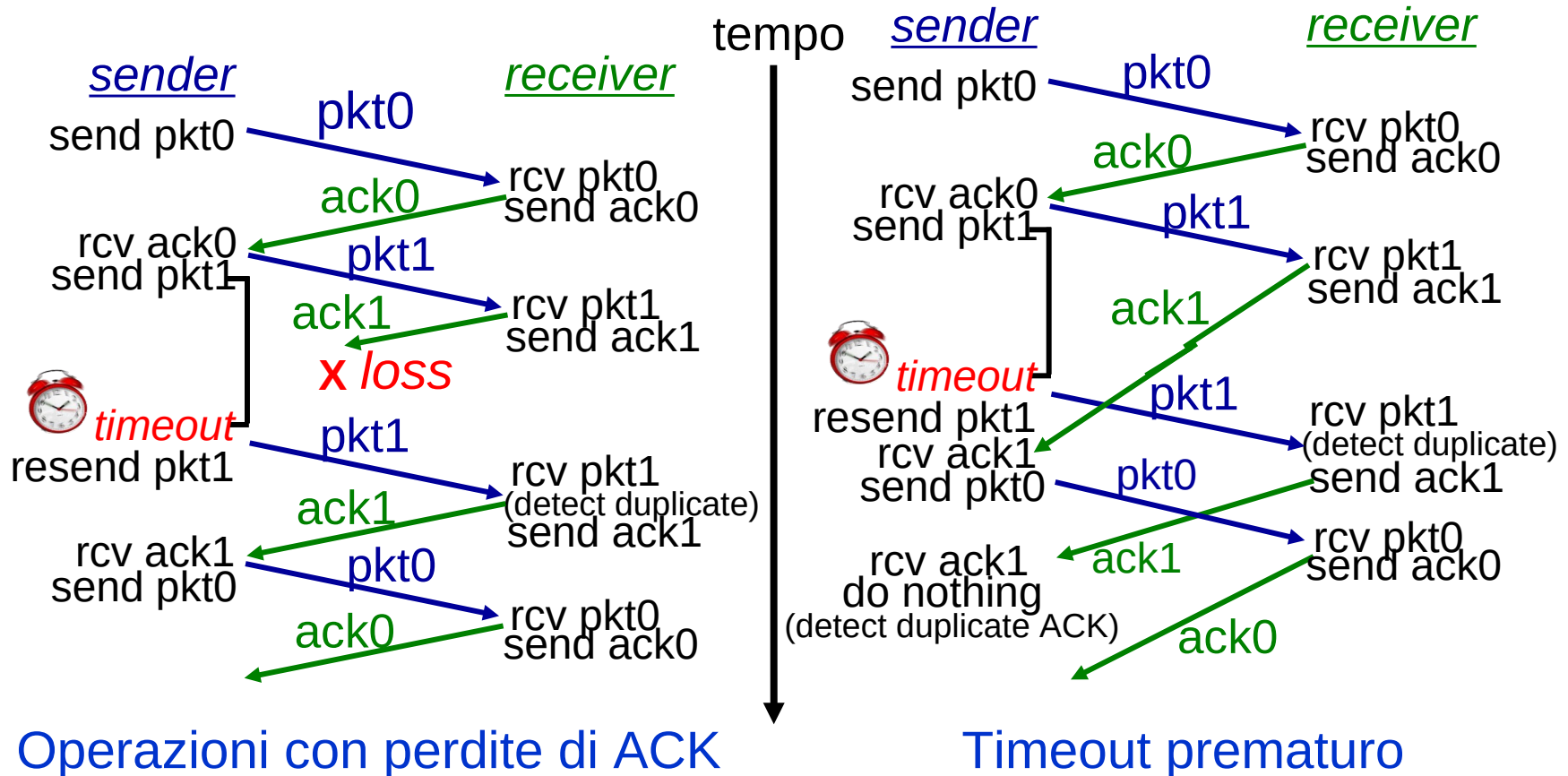
Sequenza operazioni in v3.0



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Sequenza operazioni in v3.0



Il meccanismo send-ack per singolo pacchetto è molto semplice ed affidabile, ma INEFFICIENTE

Protocollo v3.0: corretto ma inefficiente



Corretto dal punto di vista funzionale e semplice da implementare ma usa le risorse di rete e degli host in modo non efficiente → **scarse prestazioni per reti ad alta velocità**

Problema: protocollo STOP-and-WAIT

Valutiamone meglio le prestazioni: un po' di definizioni

- **Round Trip Time (RTT):** tempo impiegato da un pacchetto per andare dal mittente al destinatario e ritorno
- **Tempo di propagazione:** $RTT/2$
- **$L(pkt)$** = lunghezza pacchetto (bit)
- **Rate** = transmission rate (bit/sec)
- **Tempo di trasmissione pacchetto** = $L(pkt) / Rate$
- **Utilizzazione:** percentuale di utilizzo di una risorsa (tempo di utilizzo effettivo / tempo totale)

Tempo di trasmissione pacchetto

$$L(\text{pkt}) / \text{Rate}$$

[dimensionalmente: (bit) / (bit/sec) = sec]

Tempo di trasferimento pacchetto (vista mittente)

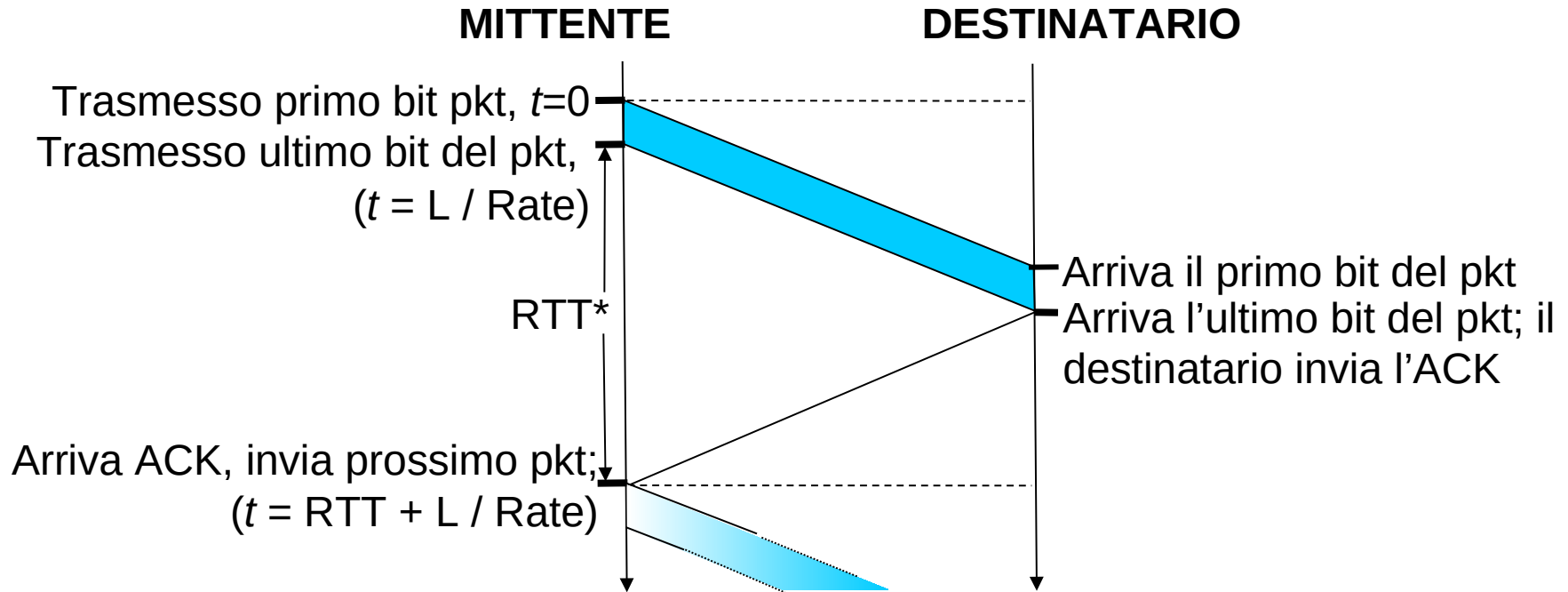
[trasmissionePkt + propagazionePkt] + [propagazioneAck]

$$= [L(\text{pkt}) / \text{Rate} + \text{RTT}/2] + [\text{RTT}/2]$$

$$= \text{RTT} + L(\text{pkt}) / \text{Rate}$$

***Assumiamo che i pacchetti ACK
siano estremamente piccoli → tempo
di trasmissione trascurabile***

Prestazioni stop-and-wait



RTT* = Round Trip Time ideale (se il destinatario non è rallentato da altri processi)

Prestazioni stop-and-wait



Esempio

- Canale fisico con transmission rate 1 Gbps ($= 10^9$ bit/sec)
- Ritardo di propagazione ($RTT/2$) = 15 msec
- Pacchetto da 1KB ($= 8$ Kb $= 8 \cdot 10^3$ bit)

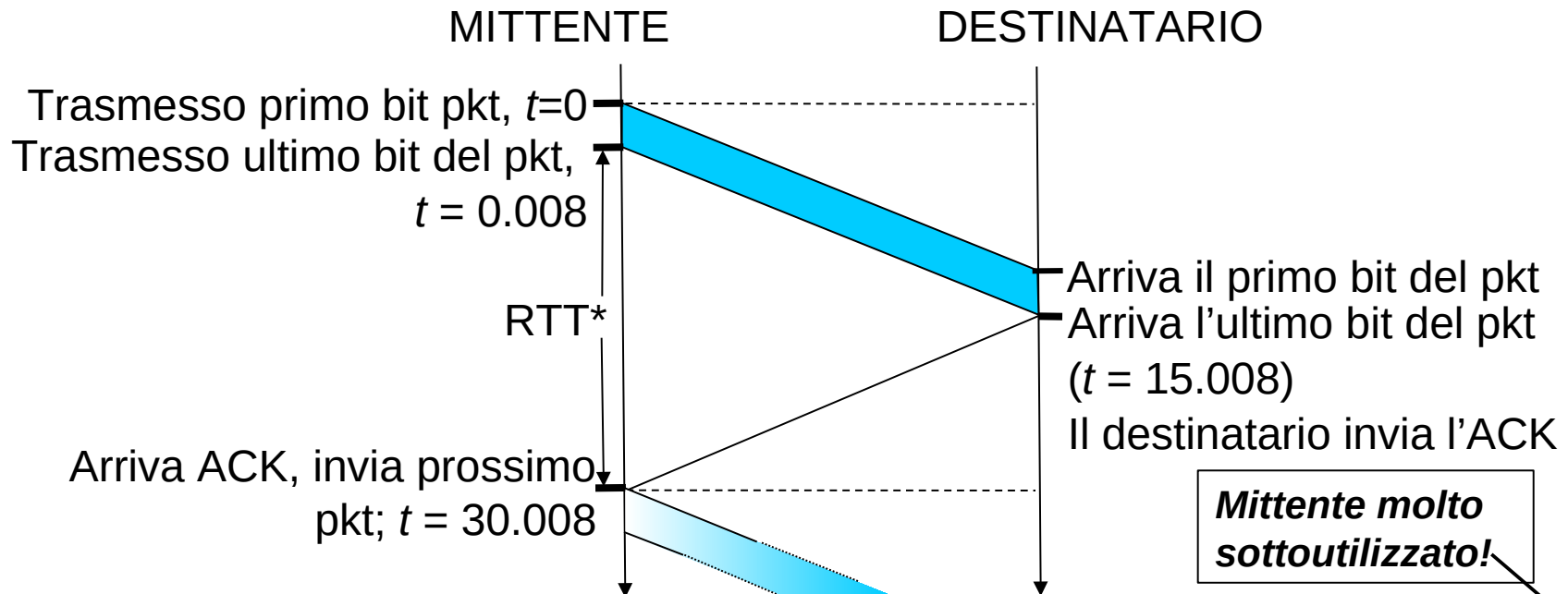
Tempo di trasmissione T del pacchetto:

$$\begin{aligned} T &= \frac{L(\text{pkt})}{\text{Rate}} = \frac{8\text{Kb}}{10^9 \text{ bit/sec}} = \frac{8 \cdot 10^3 \text{ bit}}{10^9 \text{ bit/sec}} = \\ &= \frac{8 \text{ bit}}{10^6 \text{ bit/sec}} = 0.000008 \text{ sec} = 0.008 \text{ msec} \end{aligned}$$

Prestazioni stop-and-wait



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



$$\text{Utilizzazione (mittente)} = \frac{\text{Tempo trasm.}}{\text{Tempo totale}} = \frac{T}{RTT + T} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30 + 0.008} = 0.00027$$

In pratica, si riesce a trasmettere ~1 KB (1 pkt) ogni 30 msec
→ ~264 Kbit ogni secondo, su un link da 1 Gbps!

Protocollo limita le prestazioni di rete!

➔ **Evidenza dell'uso limitato delle risorse fisiche di rete**

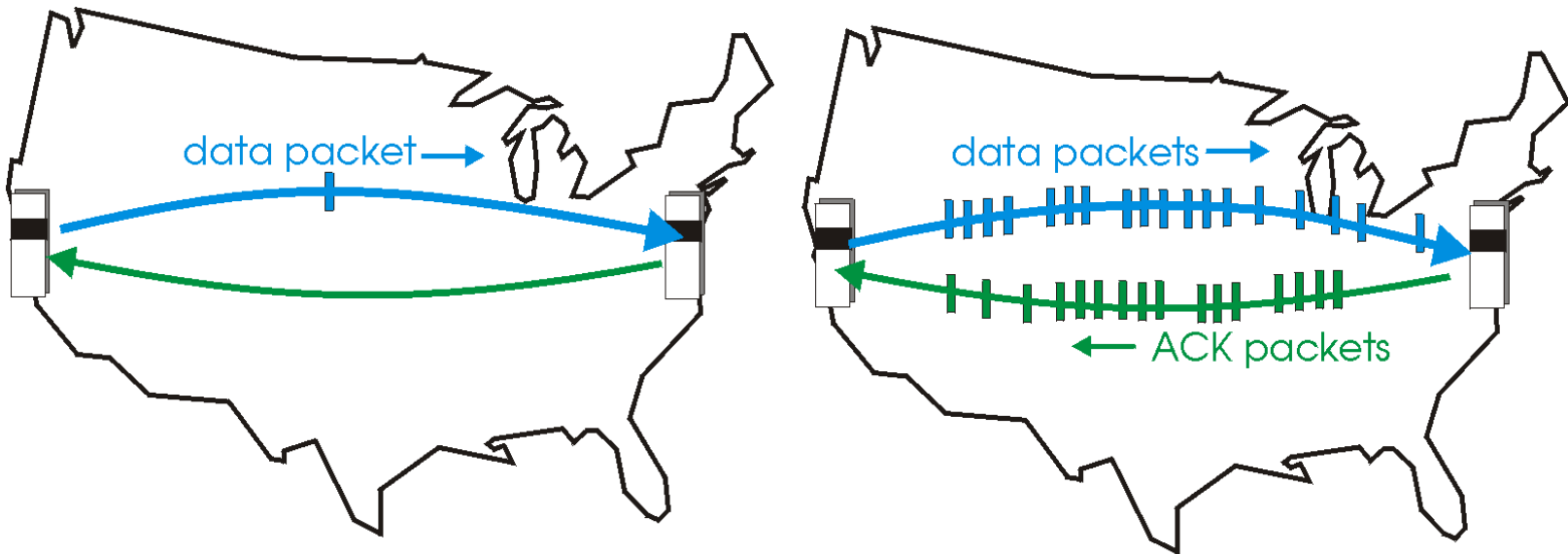
Protocolli Pipelined

Combinare affidabilità ed efficienza



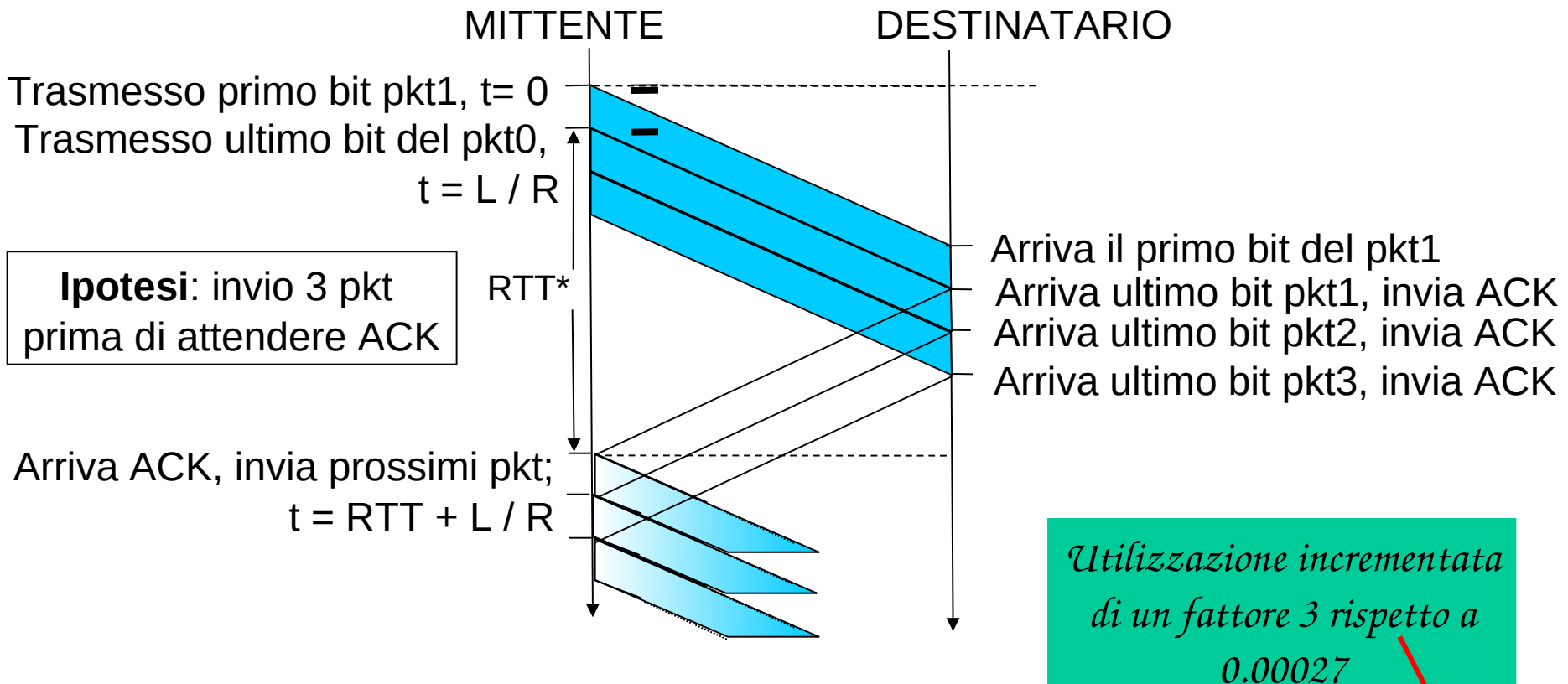
SOLUZIONE

Pipelining → il mittente invia un numero multiplo di segmenti prima di mettersi in attesa di un ACK



(a) Da protocollo stop-and-wait a protocollo pipelined

Incremento prestazioni



$$\text{Utilizzazione (mittente)} = \frac{\text{Tempo trasm.}}{\text{Tempo totale}} = \frac{T}{RTT + T} = \frac{3 \cdot L/R}{RTT + L/R} = \frac{0.024}{30 + 0.008} = 0.0008$$

Requisiti tecnici per l'implementazione



1. L'intervallo dei **numeri di sequenza** di pacchetti e ACK deve essere **incrementato** (non più sufficiente 0 e 1)
2. Necessità di un **buffer lato mittente**
 - Per mantenere i pacchetti inviati e di cui non ha ancora ricevuto ACK (potrebbero dover essere ritrasmessi)
3. Necessità di un **buffer lato destinatario**
 - Per mantenere le sequenze di pacchetti dove non tutti i pacchetti sono arrivati o sono arrivati correttamente (possibile arrivo fuori ordine)
4. Necessità di una “**finestra a scorrimento**” (***sliding window***) che denota il numero massimo di pacchetti che il mittente può inviare senza aver ricevuto un ACK dal destinatario

Vediamo intanto il concetto della sliding window...

Sliding window - mittente

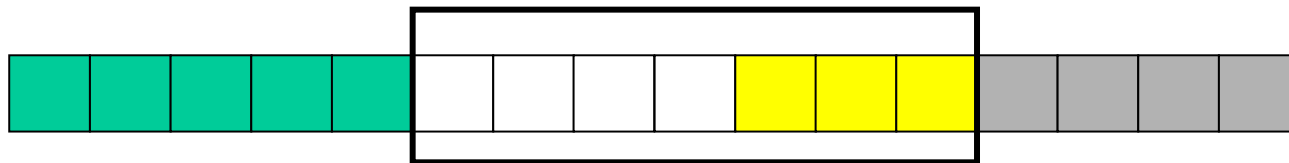


- Il mittente assegna a ciascun segmento un numero di sequenza **Num_seg**
- Ad ogni istante, il mittente mantiene una *finestra scorrevole* sugli indici dei segmenti, e **solo quelli all'interno della finestra possono essere trasmessi**
- La **dimensione della finestra** è detta N e definiamo:
 - **base**: il numero sequenza del pacchetto più vecchio che ancora non ha ricevuto ACK (inizio finestra)
 - **nextseqnum**: il numero di sequenza del primo pacchetto “inutilizzato” (inviabile ma non ancora inviato)

Sliding window - mittente

- Le tre variabili del mittente devono soddisfare la seguente relazione:
- $\text{nextseqnum} - \text{base} \leq N$

$\leq N$



*Pacchetti inviati e
confermati (ack)*

*Pacchetti
inviati*

*Pacchetti
da inviare
entro la finestra*

*Man mano che
arrivano gli ACK
dei pacchetti spediti
la finestra viene
spostata in avanti*

Algoritmi per l'affidabilità del pipeling



Ci sono **due filosofie alternative** per affrontare il problema dell'affidabilità della comunicazione in un protocollo con pipeling

1. **Go-Back-N**
2. **Selective Repeat** (ripetizione selettiva)

Quantità di numeri di sequenza e requisiti dei buffer dipendono da quale algoritmo si segue, che fondamentalemente determina **come il protocollo reagisce ai pacchetti persi**

NOTA: Sono algoritmi generali, non riferibili in modo specifico al protocollo TCP

Go-Back-N (GBN)



Go-Back-N:

- Il mittente può trasmettere al massimo **N pacchetti in pipeline** senza attendere ACK
 - N: dimensione della **sliding window**
- **ACK cumulativi**: un ACK per un pacchetto indica che tutti i pacchetti precedenti sono stati ricevuti. In caso di pacchetti fuori ordine, il destinatario invia di nuovo ACK per ultimo pacchetto ricevuto correttamente e in ordine
- Il destinatario **scarta i pacchetti ricevuti fuori ordine**
- Mittente: se scade il **timeout di un pacchetto**, ritrasmette tutti i pacchetti successivi già spediti ma per cui non ha ricevuto ACK (*da qui il nome Go-Back-N*)

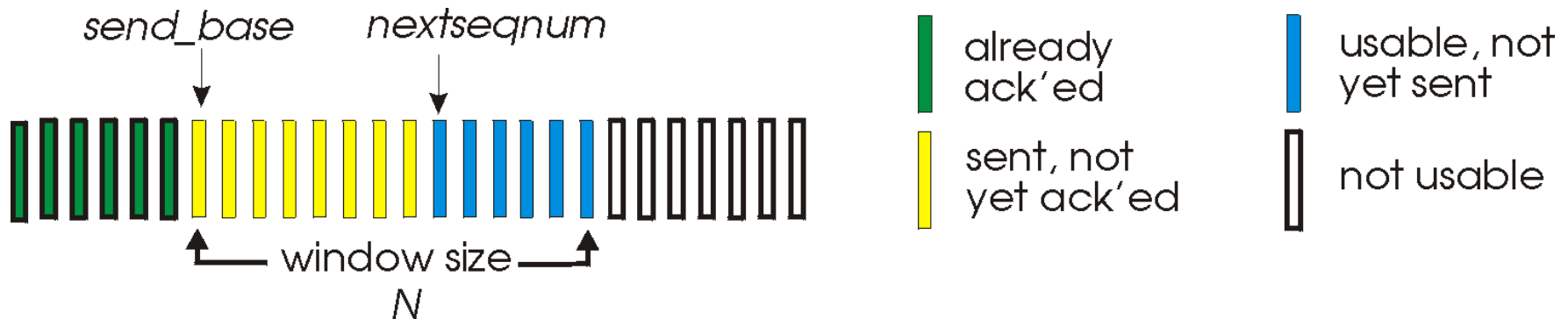
Questo meccanismo porta ad uno **spreco**: pacchetti ricevuti correttamente sono scartati

Problemi di prestazione: se nella pipeline si trovano numerosi pacchetti, un errore su un solo pacchetto può portare a numerose ritrasmissioni inutili

Go-Back-N - mittente



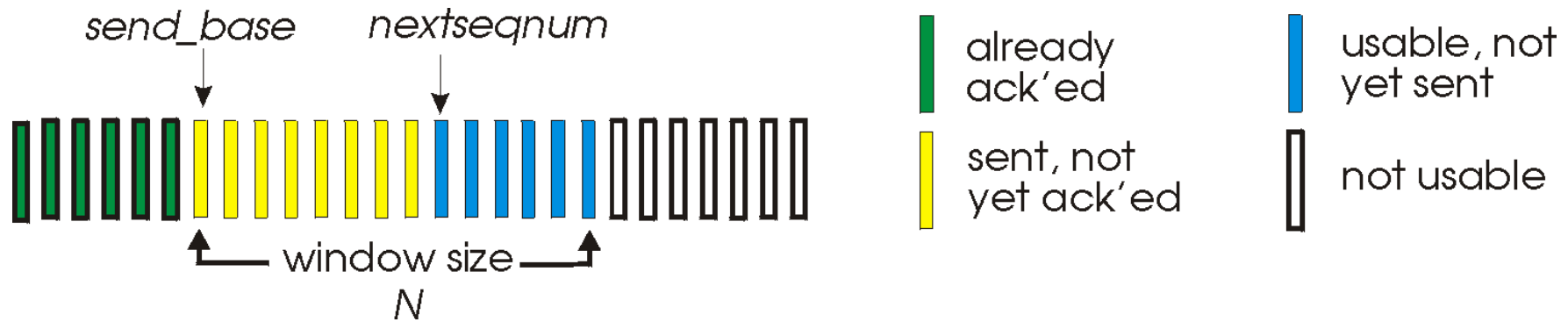
Sliding Window di max N segmenti consecutivi inviabili senza ACK



Quattro intervalli di numeri di sequenza

- $[0, \text{send_base}-1]$: pkts già trasmessi per cui si è ricevuto ACK
- $[\text{send_base}, \text{nextseqnum}-1]$: pkts inviati ma non ancora ricevuto ACK
- $[\text{nextseqnum}, \text{send_base}+N-1]$: pkts che si possono inviare immediatamente
- $[\text{send_base}+N,]$: pkts non utilizzabili al momento

Go-Back-N - mittente



- **ACK cumulativi:** se ricevo ACK per pacchetto n , so che tutti i precedenti sono arrivati correttamente
 - Posso ricevere ACK duplicati
- **Timer al mittente:** considerato per il pacchetto più 'vecchio' (spedito meno di recente) per cui non si è ricevuto ACK
- **Evento di timeout:** se scatta per un pacchetto n , ritrasmetto n e tutti e pacchetti con numero di sequenza più alto nella sliding window
- Il nome '**Go-Back-N**' deriva dal comportamento in caso di pacchetti persi: ritrasmissione di *tutti* i pacchetti spediti che non hanno ricevuto riscontro

Visione lato destinatario

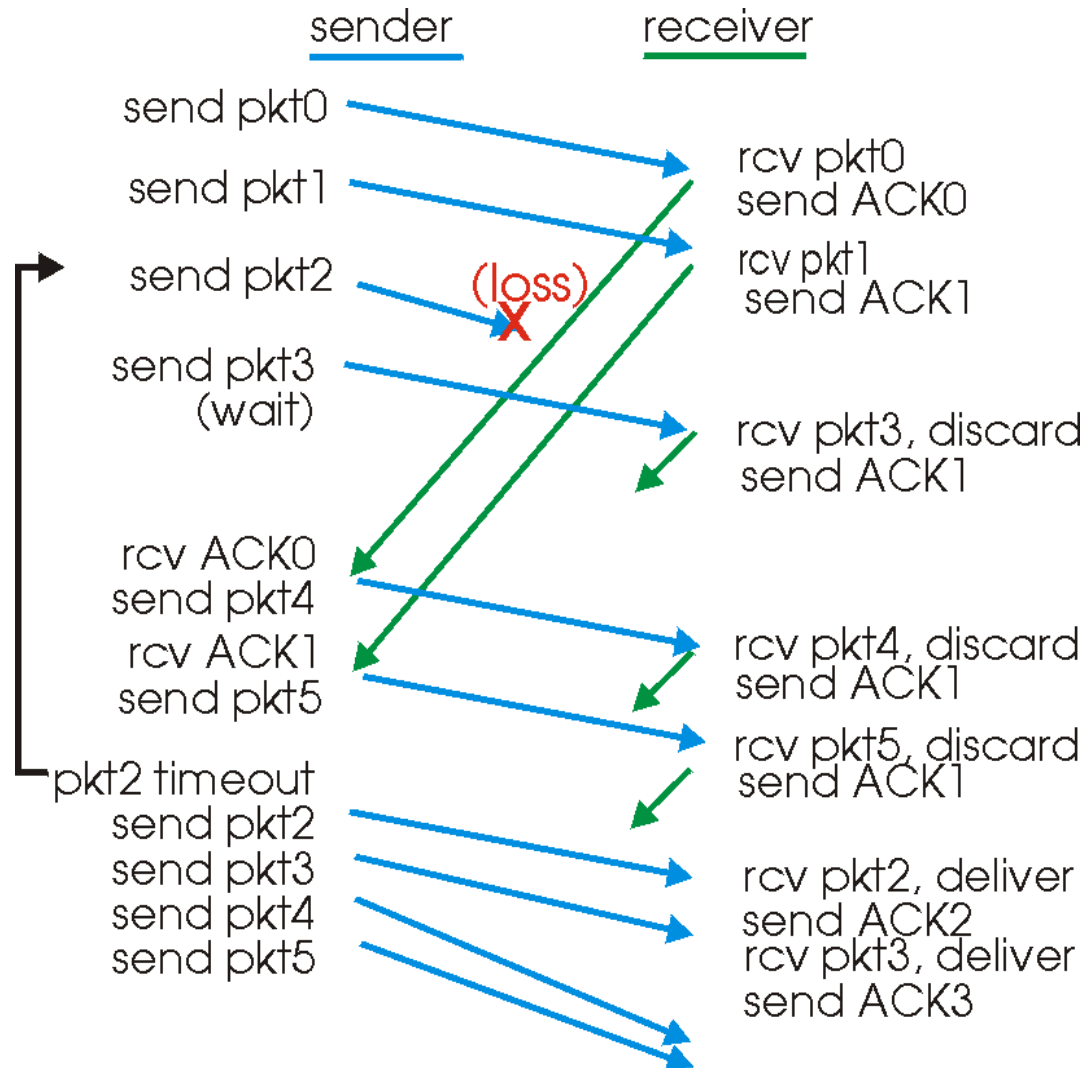
- **ACK inteso come cumulativo** da parte del destinatario → ACK del pacchetto n conferma al mittente che sono arrivati correttamente i primi n segmenti
 - E consegna il pacchetto n al livello superiore
 - **Segmenti che arrivano fuori sequenza vengono scartati** senza essere inseriti nel buffer, e il destinatario **rimanda un ACK** per il pacchetto in ordine ricevuto più di recente
 - *Perchè scartare pacchetti ricevuti correttamente?*
 - **Semplicità:** evitare gestione buffer destinatario, unica informazione memorizzata = num. di sequenza del pacchetto successivo di cui si è in attesa
 - Verranno rispediti dal mittente per il funzionamento del GBN
- **Non c'è bisogno di buffer di ricezione** a lato destinatario per gestire il pipeling

Funzionamento GBN



Ipotesi:

Finestra
 $N=4$



Go-Back-N vs. Selective Repeat



Go-Back-N:

- Il mittente può trasmettere al massimo **N pacchetti in pipeline** senza attendere ACK
 - N: dimensione della **sliding window**
- **ACK cumulativi**: un ACK per un pacchetto indica che tutti i pacchetti precedenti sono stati ricevuti
- Il destinatario **scarta i pacchetti ricevuti fuori ordine**
- Mittente: se scade il **timeout di un pacchetto**, ritrasmette tutti i pacchetti successivi già spediti ma per cui non ha ricevuto ACK (*da qui il nome Go-Back-N*)

Selective Repeat:

- Il mittente può trasmettere al massimo **N pacchetti in pipeline** senza attendere ACK
- Il destinatario invia **ACK individuali** per ogni pacchetto ricevuto correttamente (anche se i precedenti non sono stati ricevuti correttamente)
- Mittente: mantiene un **timer per ogni pacchetto senza ACK**
 - Quando il timeout scade, **ritrasmette solo** il pacchetto per cui è scaduto (da qui il nome **ripetizione selettiva**)

Selective Repeat



Il destinatario invia ACK di tutti i segmenti ricevuti correttamente

- Necessaria bufferizzazione dei pacchetti, per consegnarli secondo la sequenza ordinata al processo applicativo

Il mittente ritrasmette soltanto i segmenti per i quali non ha ricevuto ACK dal destinatario entro il time-out

- Utilizza timer individuali

Sia mittente che destinatario gestiscono due finestre di sequenze di N segmenti consecutivi

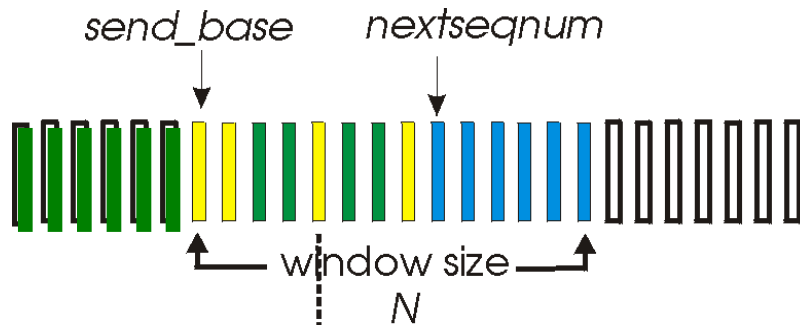
- La finestra del mittente avanza di 1 quando riceve un ACK entro il timeout sul segmento `send_base`
- La finestra del destinatario avanza quando il segmento `rcv_base` è ricevuto correttamente

Finestre mittente e destinatario



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Ipotesi: finestra di dimensione $N = 14$



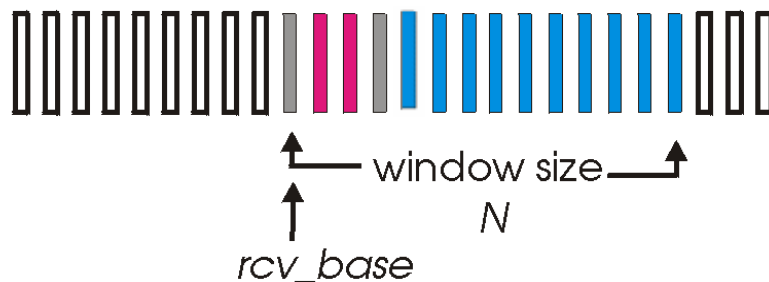
already
ack'd

sent, not
yet ack'd

usable, not
yet sent

not usable

Sliding window del mittente



out of order
(buffered) but
already ack'd

Expected, not
yet received

acceptable
(within window)

not usable

Sliding window del destinatario

Algoritmo Selective Repeat



mittente

Dati dall'applicazione:

- se c'è un numero di sequenza 'usabile' nella finestra, invia il segmento

Timeout(i):

- ritrasmetti segmento i e inizializza nuovo timer per i

ACK(i) nella finestra

[send_base, send_base+N]:

- segna segmento i ricevuto
- se i è il segmento "base" non ancora ACKed, incrementa send_base fino al successivo pacchetto non ACKed

destinatario

Segmento i ricevuto in

[rcv_base, rcv_base+N-1]

- invia ACK(i)
- *non ordinato*: metti in buffer
- *ordinato*: consegna al processo applicativo e; avanza rcv_base fino al segmento successivo non ancora ricevuto

Segmento i ricevuto in

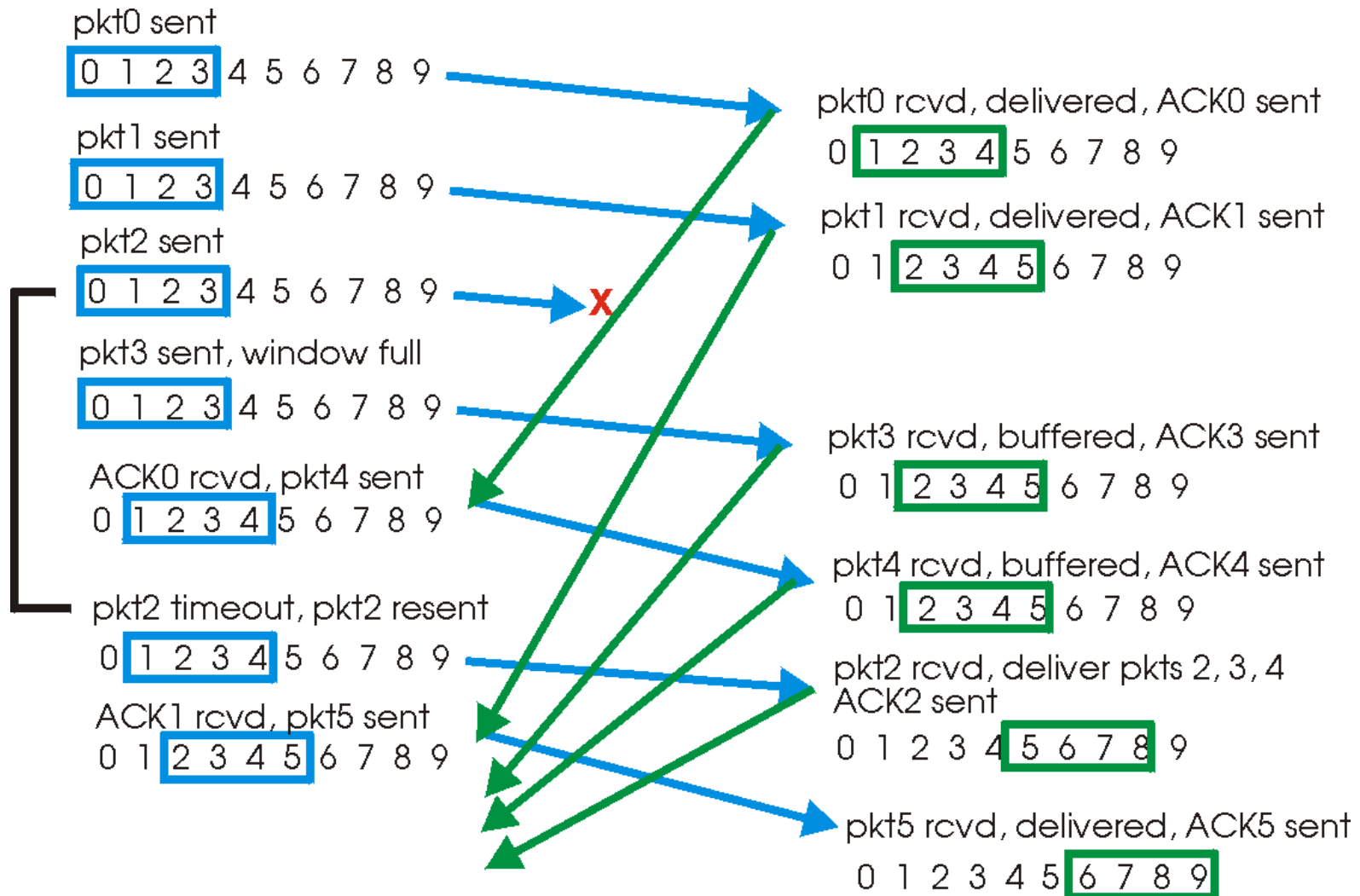
[rcv_base-N, rcv_base-1]

- ACK(i)

Altrimenti – fuori finestra:

- ignora

Funzionamento Selective Repeat



Trasporto orientato alla connessione: TCP

TCP (Transmission Control Protocol)



- Protocollo che fornisce un livello di **trasporto affidabile ed orientato alla connessione**
- Servizi **aggiuntivi rispetto a UDP**
 - **orientato alla connessione**: comprende le fasi di instaurazione (*invio reciproco di segmenti per stabilire i parametri del successivo trasferimento*), utilizzo e chiusura della connessione
 - **connessione full duplex (bi-direzionale)**: una volta instaurata una connessione, è possibile il trasferimento contemporaneo in entrambe le direzioni della connessione
 - **orientato al flusso di dati**: considera il flusso di dati dall'host mittente fino al destinatario (sia attraverso la rete che agli host terminali)
 - **trasferimento con buffer**: i dati da spedire sono memorizzati in un buffer di invio e da lì prelevati a blocchi e spediti

Cosa il TCP non garantisce



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Comunicazioni in tempo reale**
- **Garanzia di disponibilità di banda** tra mittente e destinatario
- **Multicast** (un mittente, molti destinatari)
 - Per questo si usano protocolli senza connessione come UDP

Problemi da affrontare a livello TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Eterogeneità degli host** e dei processi in comunicazione
 - Bisogno di un meccanismo per attivare e concludere una comunicazione in modo esplicito per entrambi
- **Eterogeneità dei tempi di trasmissione**
 - Bisogno di un meccanismo di **timeout adattativo**
- **Possibilità di ritardi molto lunghi nella rete**
 - Bisogno di gestire il possibile arrivo di pacchetti molto vecchi
- Possibilità di avere un host destinatario con **capacità molto diversa** dall'host mittente
 - Bisogno di gestire capacità dei nodi eterogenee
- Possibilità di avere **capacità di rete molto diverse**
 - Bisogno di gestire possibili congestioni dovute alla rete

Orientato alle connessioni



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Connection oriented** significa che:
 - viene creata una connessione tra i due host prima del trasferimento di qualunque dato tra le applicazioni → in modo trasparente per l'applicazione
 - viene chiusa dopo il completamento del trasferimento dati
- **3 fasi**
 - Instaurazione
 - Utilizzo
 - Chiusura
- **Il processo applicativo viene avvisato solo se:**
 - non si riesce a stabilire la connessione
 - la connessione viene interrotta

Trasmissione byte stream



- **Byte stream** (orientata al flusso di dati) significa che la connessione viene trattata come un **flusso di byte continuo** dal mittente al destinatario
- L'**unità di trasmissione** nello stream è il byte
 - Il processo applicativo mittente scrive byte
 - Il livello TCP, per inviarli, accorpa i byte in un segmento TCP
 - Il livello IP incapsula ogni segmento TCP in un datagram IP
- Il processo applicativo destinatario legge byte

Connessione full-duplex



- TCP può effettuare **trasferimenti contemporanei** in **entrambe le direzioni** della connessione, nell'ambito della stessa sessione
- Ai processi applicativi, questi trasferimenti “appaiono” come **due data stream non correlati**
- Tuttavia, TCP consente di **sovrapporre (piggybacking)** comunicazioni di dati e comunicazioni di controllo, con l'invio di informazioni di controllo (es., ACK) insieme ai dati utente

- **Affidabile** → TCP gestisce un **trasferimento ordinato** di uno stream di dati:
- **acknowledgment + time-out+(ritrasmissione)**
 - Ogni trasmissione andata a buon fine viene notificata (**acknowledged**) dall'host ricevente
 - Se l'host mittente non riceve un acknowledgement entro un intervallo di tempo predefinito (**time-out**), il mittente **ritrasmette** i dati
 - Acknowledgment e ritrasmissioni dovute ad eventuali perdite sono gestite in modo **trasparente rispetto al processo applicativo**

Trasmissione con buffer

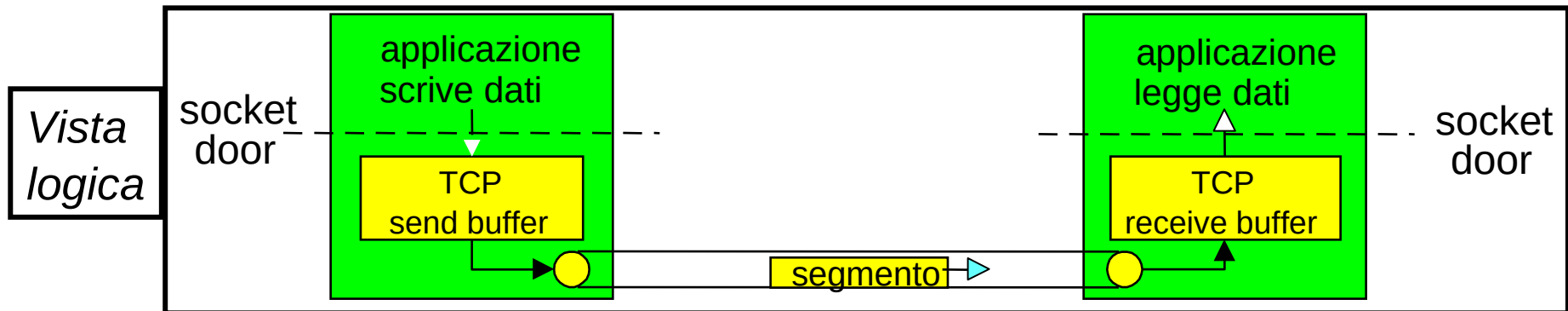


- Per far fronte a molti dei problemi evidenziati in precedenza, il layer TCP deve utilizzare necessariamente dei **buffer**
- I buffer consentono di ovviare a:
 - tempi di trasmissione differenti tra host
 - capacità di invio e di ricezione differenti
 - segmenti persi o fuori ordine
- Il livello **TCP** è **responsabile della gestione del buffering** dei dati e di determinare quando è tempo di inviare un certo insieme di dati. Alcune scelte:
 - **congestion control**: l'host mittente deve diminuire il tasso di trasmissione dei pacchetti quando la rete è congestionata
 - **flow control**: l'host mittente non deve sovraccaricare l'host ricevente

Massima dimensione segmento



- Massima dimensione di dati in un segmento → **Maximum Segment Size (MSS)**: concordato tra mittente e destinatario oppure il default dipende dall'implementazione TCP (es., 1460 byte, 536 byte, 512 byte) o dal **MTU (Maximum Transmission Unit)** ossia la lunghezza del frame più grande che può essere trasmesso a livello di collegamento dall'host mittente (evitare frammentazione H2N)
- Es. Ethernet MTU = 1500 byte → MSS 1460 byte (header TCP ~40byte)
 - Approcci euristici proposti per scoprire la MTU lungo un percorso



Il segmento TCP

Segmento TCP

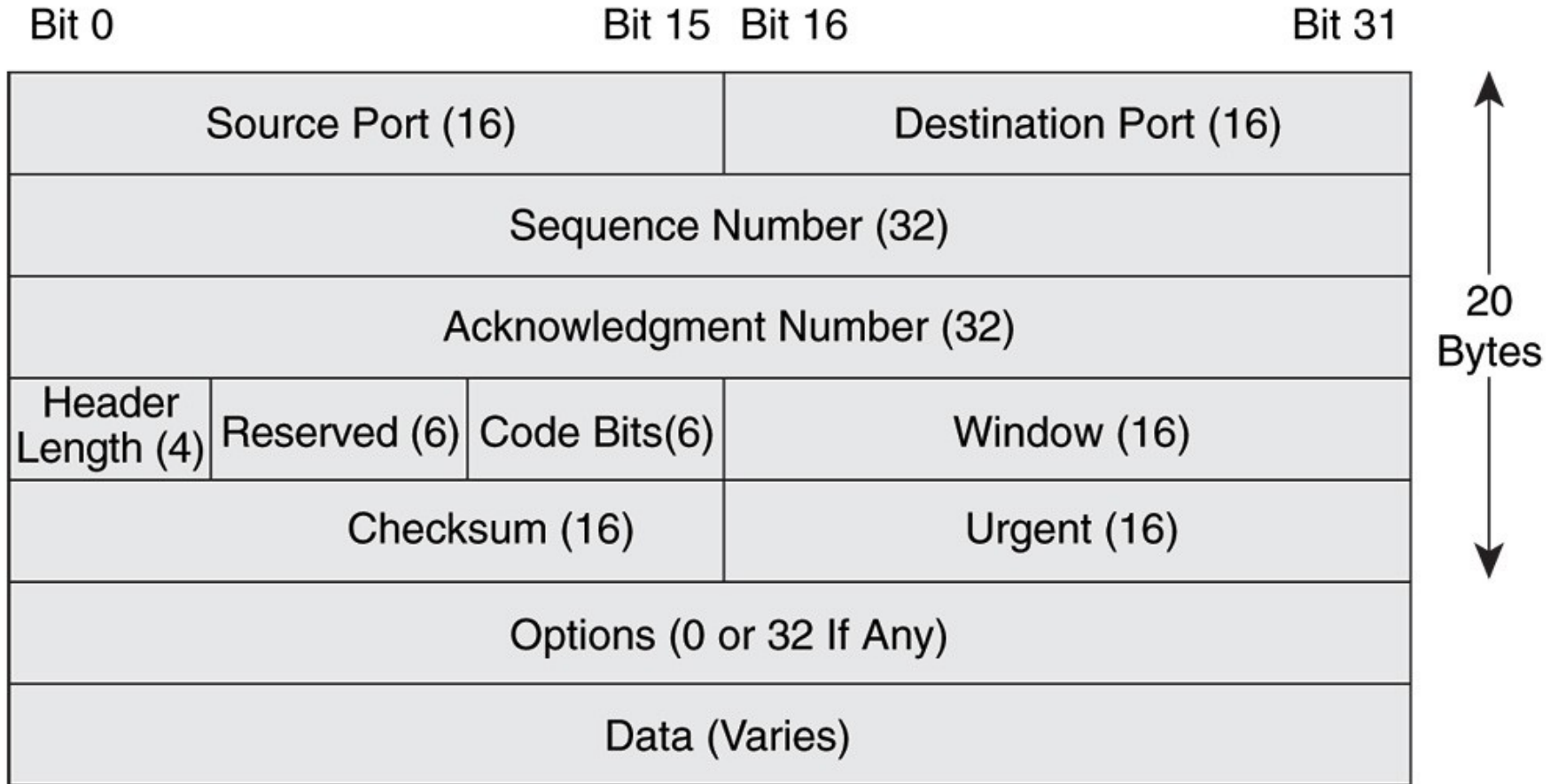


- TCP accoppia ad ogni blocco dati da spedire una intestazione TCP andando pertanto a formare un **segmento TCP**
- Ogni segmento viene **passato al livello di rete sottostante** dove viene **incapsulato separatamente in un datagramma IP** e spedito
- Quando ricevuti all'altro lato vengono inseriti nel buffer di ricezione del destinatario da cui l'applicazione li legge
- **Ogni segmento TCP contiene:**
 - **Header:** informazioni di controllo per identificare i byte dati
 - **Payload:** dati del byte stream (limitato da MSS)
- Per spedire un file di grandi dimensioni, TCP lo frammenta in porzioni di dimensione MSS (l'ultima sarà più piccola)

Formato del segmento TCP



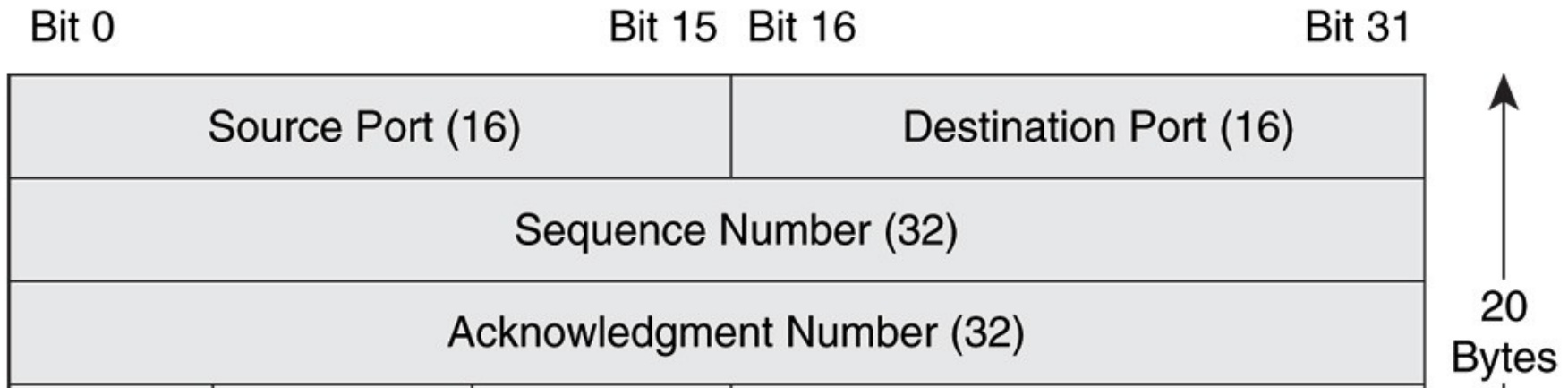
UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Formato del segmento TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

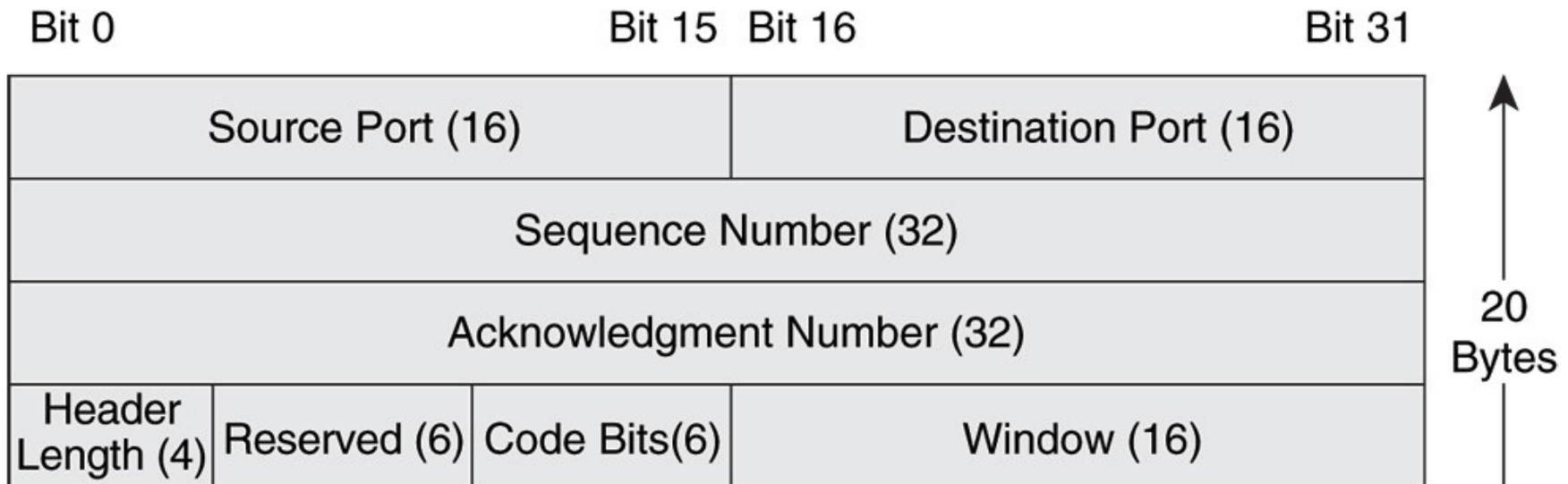


- **source port (16 bit):** numero di porta del mittente
- **destination port (16 bit):** numero di porta del destinatario
- **sequence number (32 bit):** numero di sequenza relativo al flusso di byte che si sta trasmettendo
- **acknowledgement number (32 bit):** ACK relativo ad un numero di sequenza del flusso di byte che si sta ricevendo (il flusso è bi-direzionale, vi è la possibilità di *piggybacking* → si vedrà in seguito)

Formato del segmento TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



- **hlen** (4 bit): lunghezza dell'header TCP (in multipli di 32 bit) se non vi sono opzioni (campo opzioni vuoto) → hlen = 20 byte
- **reserved** (6 bit): per usi futuri
- **code bits – o flag** (6 bit): indicano scopo e contenuto del segmento
 - **URG** (urgent): dati segnati come urgenti dal livello applicativo
 - **ACK** (acknowledgement): valore nel campo acknowledgement è valido
 - **PSH** (push): il destinatario deve passare i dati all'applicazione immediatamente
 - **SYN** (synchronize), **FIN**, **RST** (reset): usati per instaurazione, chiusura ed interruzione della connessione

Formato del segmento TCP



- **window size** (16 bit): dimensione della finestra in ricezione (numero di byte che si è disposti ad accettare in ricezione → **controllo di flusso!**)
- **checksum** (16 bit): controllo integrità dei dati trasportati nel segmento TCP (del tutto analogo al caso del protocollo UDP)
- **urgent pointer** (16 bit): puntatore al termine dei dati urgenti (usato molto raramente - come PSH e URG - nel caso di trasmissione di caratteri speciali)
- **TCP options**: campo opzionale di lunghezza variabile (serve a negoziare la dimensione del *segmento massimo scambiato* → **MSS**)

Header Length (4)	Reserved (6)	Code Bits(6)	Window (16)
Checksum (16)			Urgent (16)
Options (0 or 32 If Any)			
Data (Varies)			



Checksum TCP



- Utilizzato per rilevazione errori nei dati trasportati:
- Calcolato usando un maggior numero di informazioni di quelle presenti nell'header TCP (vi sono anche informazioni IP)
→ **definizione di uno *pseudo-header* TCP**

32 bit

indirizzo IP mittente		
indirizzo IP destinatario		
zero padding	protocollo	lunghezza TCP

- *zero padding*: dimensione dello pseudo-header, multiplo di 32 bit
- *protocollo*: campo protocollo del datagram IP
- pseudo-header anteposto al segmento TCP
- checksum calcolato su pseudo-header e intero segmento TCP
- pseudo-header *non* è trasmesso dal mittente

Checksum - dubbi



- Il campo checksum può essere corrotto? *Sì, ma...*
 - Se sia il checksum che i dati sono corrotti, è altamente improbabile che il controllo dia esito positivo
- Il meccanismo del checksum garantisce la rilevazione di errori su singoli bit (1 bit per pacchetto)
- Il checksum fallisce nel rilevare errori relativi allo scambio di due byte (byte swap)
- Tuttavia, le linee reali hanno caratteristiche (nella produzione di errori) molto più comuni di altre
 - Azzeramento di tutti i bit di un 'pezzo' di dati → **Molto comune**
 - Byte swap → **praticamente impossibile!**

Negoziiazione MSS



- Le TCP OPTIONS consentono di **negoziare il Maximum Segment Size (MSS)** per
 - evitare il più possibile la frammentazione ai livelli sottostanti
 - sfruttare al meglio la banda
 - **Trade-off**
 - MSS troppo piccolo → **overhead** eccessivo dovuto agli header
 - MSS troppo grande → elevati rischi di **frammentazione** nell'attraversamento dei livelli dello stack sottostanti IP-H2N
- Default MSS = 536 byte (il minimo possibile)
- Spesso in negoziazione si usa MSS = 1460 byte
- **Rischi frammentazione:** l'ACK viene mandato a livello del segmento originale, quindi se un frammento viene perso, tutto il segmento deve essere reinviato

Instaurare e chiudere una connessione TCP

Instaurazione di una connessione



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Modello client/server**
 - client: inizia la connessione
 - server: per accettare la richiesta di instaurazione della connessione, il server deve essere già attivo e in attesa del contatto dal client
- **Inizializzazione delle variabili del TCP:**
 - numeri di sequenza dei segmenti
 - informazioni necessarie per la gestione del buffer di trasmissione e ricezione
- Per **richiedere una connessione**, il client invia un segmento TCP speciale, detto “**SYN**” (synchronize) al server
 - Flag SYN settato nel campo Code bits (flag)
- Seguito da un pacchetto SYN del server (detto anche SYNACK)
- Seguito da un pacchetto ACK del client

Segmento SYN del client



Il segmento SYN del client include:

- **Initial Sequence Number (ISN) del client:**
 - Un numero di 32 bit
 - Il numero è scelto in modo **pseudo-casuale** tra 0 e $2^{32}-1$
 - Se il numero iniziale è 2032 e ci sono da spedire 5000 byte, tutti i byte saranno numerati da 2032 a 7031
 - Comunicazione full duplex: la numerazione dei byte in una direzione della comunicazione è **indipendente** dalla numerazione dei byte nell'altra direzione
 - Nell'**intestazione di ogni segmento** è riportato solo il numero di sequenza del **primo byte dei dati** contenuto nel segmento: gli altri byte del segmento sono numerati di conseguenza

Segmento SYN del client



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Il segmento SYN del client include anche:

- **Maximum Receive Window (MRW) del client:** il massimo numero di byte che il client è in grado di ricevere nel suo buffer – necessario per la **regolazione del flusso** dello stream di byte server → client
- **Maximum Segment Size (MSS):** la massima dimensione del segmento (informazione non sempre inviata)
- Ha il bit SYN posto a 1 nei flag

Il segmento SYN NON HA payload (dati del messaggio),
ma solo il TCP header

Segmento SYN del server



Il segmento SYN del server include:

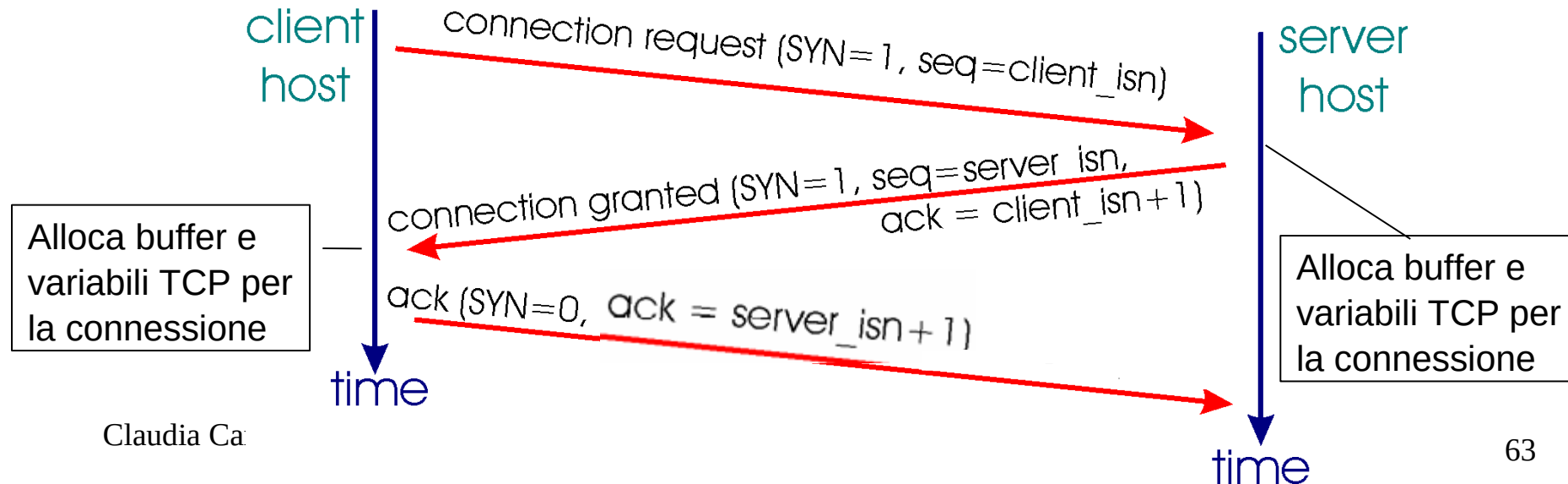
- **ACK** del server: $\text{client_ISN} + 1$
- **Initial Sequence Number (ISN)** del server: numero pseudo-casuale
 - Per la comunicazione dati server $\rightarrow$ client
- **Maximum Receive Window (MRW)** del server
 - Per la comunicazione client $\rightarrow$ server
- **Maximum Segment Size (MSS)**: la massima dimensione del segmento (informazione non sempre inviata)
- Ha il bit SYN posto a 1

NON HA payload ma solo il TCP header

Three-way handshake



- Il client invia al server un segmento di controllo con $\text{SYN}=1$, e specifica nello stesso segmento il proprio numero iniziale di sequenza (client_isn)
- Il server riceve il segmento del client con $\text{SYN}=1$
- Se accetta, il server invia un segmento di controllo con $\text{SYN}=1$, $\text{ACK}=\text{client_isn}+1$, ed il proprio numero iniziale di sequenza (server_isn)
- Il client segnala la definitiva apertura della connessione inviando un segmento di controllo con $\text{SYN}=0$ e $\text{ACK}=\text{server_isn}+1$



Three-way handshaking



In parole povere...

Client: “Hello server, voglio parlare con te, e comincerò con il byte che indicherò con il numero X ”

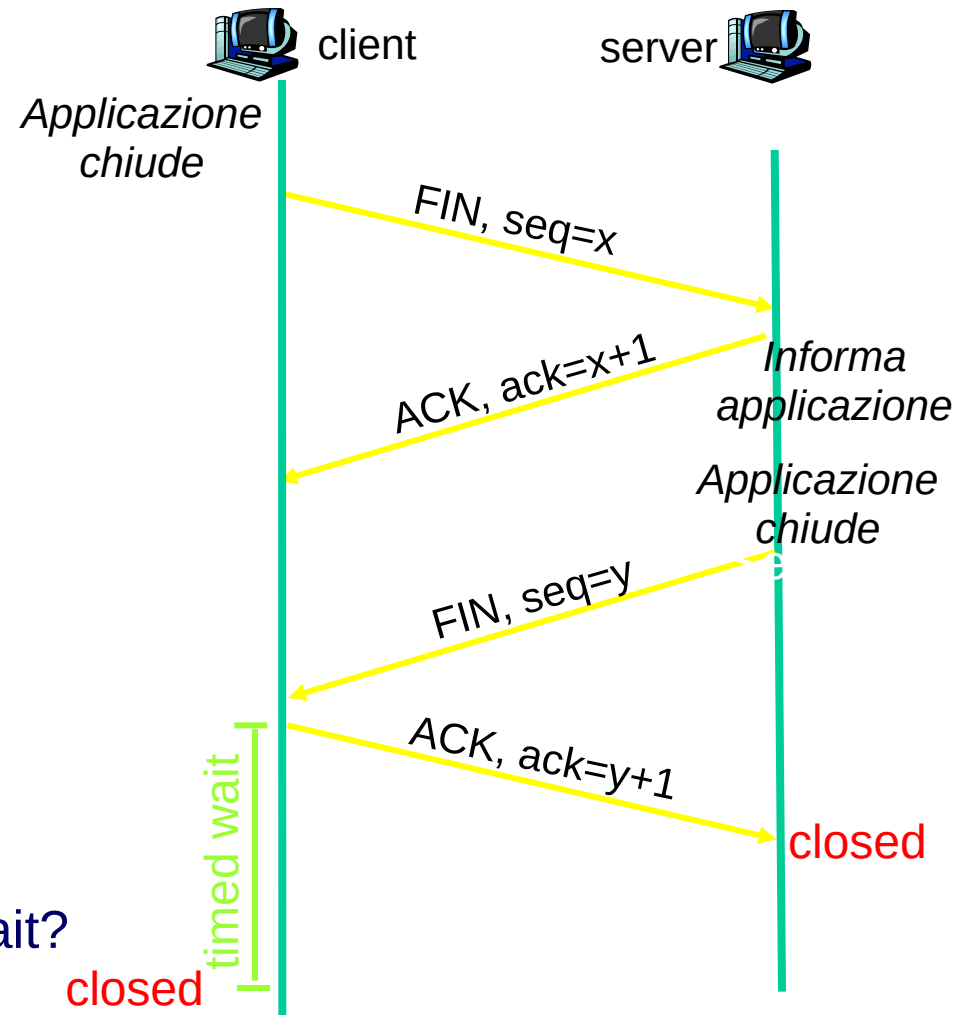
Server: “OK, sono disponibile a parlare. Il mio primo byte sarà indicato con il numero Y e so che il tuo prossimo byte sarà indicato con $X+1$ ”

Client: “D’accordo, so che il prossimo byte che mi invierai sarà indicato con il numero $Y+1$ ”

3 fasi: necessarie per avere
la sicurezza che entrambi siano pronti

Chiusura (polite) della connessione

- Il client invia un segmento di controllo con flag bit **FIN=1** al server
- Il server riceve FIN, invia ACK
- Il server chiude la connessione lato client-server ed invia **FIN=1** al client
- Il client riceve il segmento con FIN=1 ed invia ACK
- Il server riceve ACK
- Il client attende un certo tempo dopo l'invio dell'ACK; allo scadere anche la connessione lato client viene chiusa



Perché attendere anche il time wait?

Chiusura (polite) della connessione

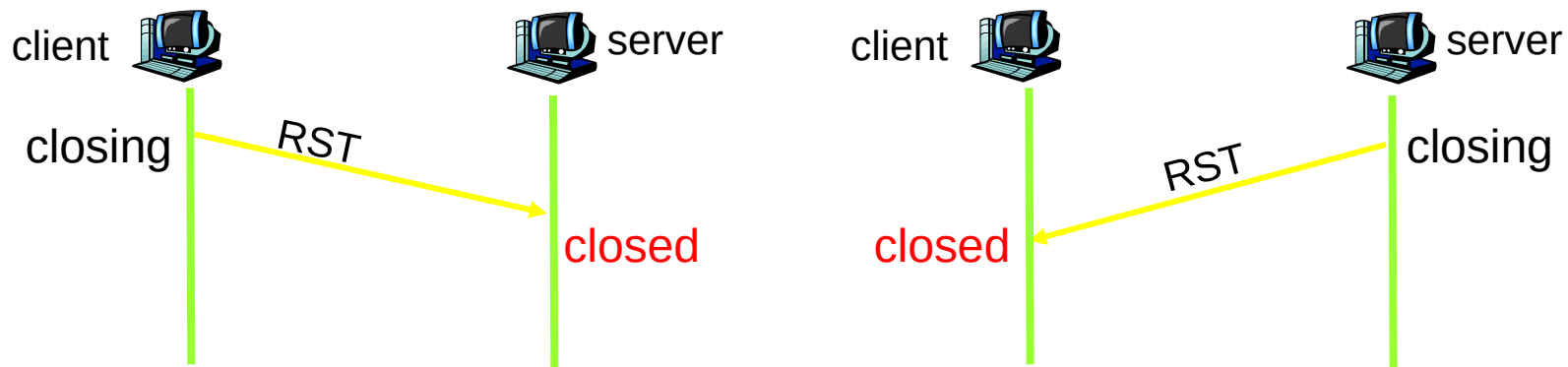


- Dopo che la connessione TCP è stata chiusa (ovvero è stato inviato l'ultimo ACK dal client), ci potrebbe essere ancora **qualcosa da fare**.
Per esempio:
 - **Se l'ultimo ACK si perde?**
 - L'ultimo segmento FIN verrà inviato nuovamente da parte del server, e dovrà essere notificato con un ACK
 - **Se segmenti persi o duplicati dovessero raggiungere la destinazione dopo un lungo ritardo?**
- Quindi, il client TCP attende per un tempo **TIME_WAIT** (es., 30 secondi) prima di chiudere definitivamente la connessione per poter gestire queste **situazioni anomale**

Chiusura (reset) della connessione



- In condizioni normali, la connessione è chiusa in **modo “polite”**
- A volte si verificano condizioni che richiedono di interrompere la connessione in **modo “brusco”** o **lato server** (tipicamente, per errori o sovraccarico) o **lato client** (errori)
- Meccanismo per la **chiusura rapida** → **reset**
 - L’host che decide il reset invia un messaggio con flag **RST=1**
 - **L’altro nodo chiude immediatamente la connessione**
 - Vengono **rilasciate tutte le risorse utilizzate** dalla connessione
- Flag RST usato anche nel caso che un server riceva segmento TCP SYN sulla porta 80 ma non abbia server attivi in ascolto su quella porta
 - Manda al mittente un segmento speciale con RST = 1



Esempio Nmap



Nmap: strumento scansione porte (*tipicamente usato dagli attaccanti per scoprire servizi attivi – quindi non concesso da host remoti*)

Per motivi di test è concesso fare scan porte su scanme.nmap.org

`nmap -v scanme.nmap.org #increase verbosity level`

Starting Nmap 7.60 (<https://nmap.org>) at 2021-12-17 10:37 CEST

Initiating Ping Scan at 10:37

Completed Ping Scan at 10:37, 0.21s elapsed (1 total hosts)

Initiating Parallel DNS resolution of 1 host. at 10:37

Initiating Connect Scan at 10:37

Scanning scanme.nmap.org (45.33.32.156) [1000 ports]

Discovered open port 80/tcp on 45.33.32.156

Discovered open port 22/tcp on 45.33.32.156

Discovered open port 9929/tcp on 45.33.32.156

PORT	STATE	SERVICE
------	-------	---------

22/tcp	open	ssh
--------	------	-----

80/tcp	open	http
--------	------	------

9929/tcp	open	nping-echo
----------	------	------------

By default, Nmap scans the most common 1000 ports for each protocol
Possibility to specify different ports for different protocols

Nmap – situazioni possibili con TCP



Per analizzare **su un host remoto una specifica porta TCP (es. 6789)** nmap manda un **segmento TCP SYN** a quell'host e quella porta

3 possibili risultati

1) Riceve segmento TCP SYNACK dall'host bersaglio

- Applicazione in ascolto su quella porta → open port

2) Riceve segmento TCP RST dall'host bersaglio

- Il segmento ha raggiunto l'host bersaglio ma non è in esecuzione alcuna applicazione che usa la porta 6789
- L'attaccante sa che la comunicazione non è bloccata da firewall sul percorso

3) Non riceve nulla

- Segmento SYN probabilmente bloccato da un firewall e non ha raggiunto il bersaglio

TCP: gestione dell'affidabilità

- **Acknowledgment (positivo)**
- **Time-out**
- **Ritrasmissione**

Uso di pipeling

Gestiti dal livello TCP in modo trasparente rispetto al processo applicativo

Principi di base:

- Ogni trasmissione andata a buon fine viene notificata (acknowledged) dall'host ricevente
- Se l'host mittente non riceve un acknowledgement entro il time-out, il mittente ritrasmette i dati

Algoritmo seguito

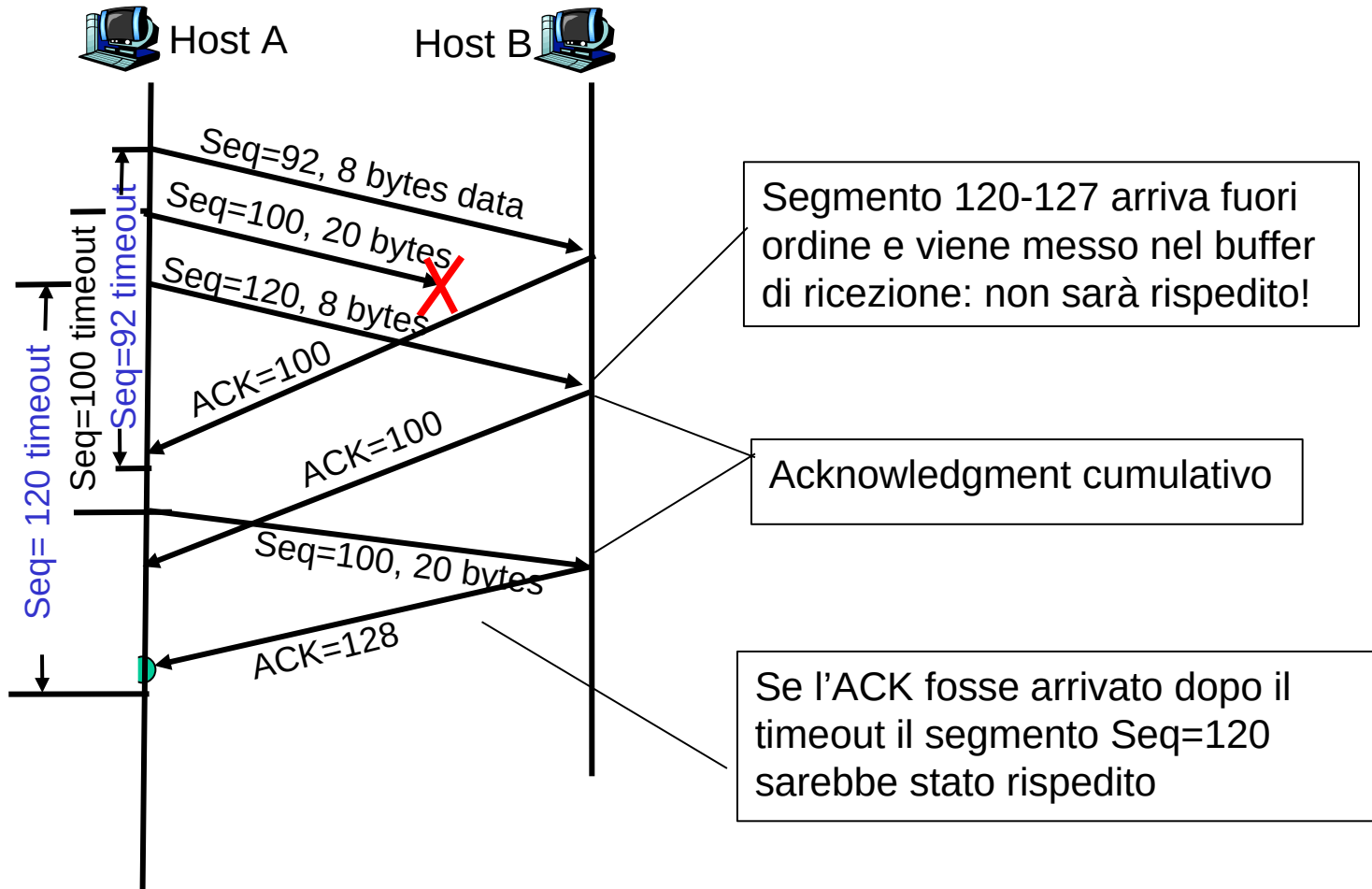
- Attualmente il **TCP non segue nessuna delle due versioni (Go-Back-N e Ritrasmissione Selettiva) in modo puro, ma un insieme delle due**

Algoritmo del protocollo TCP

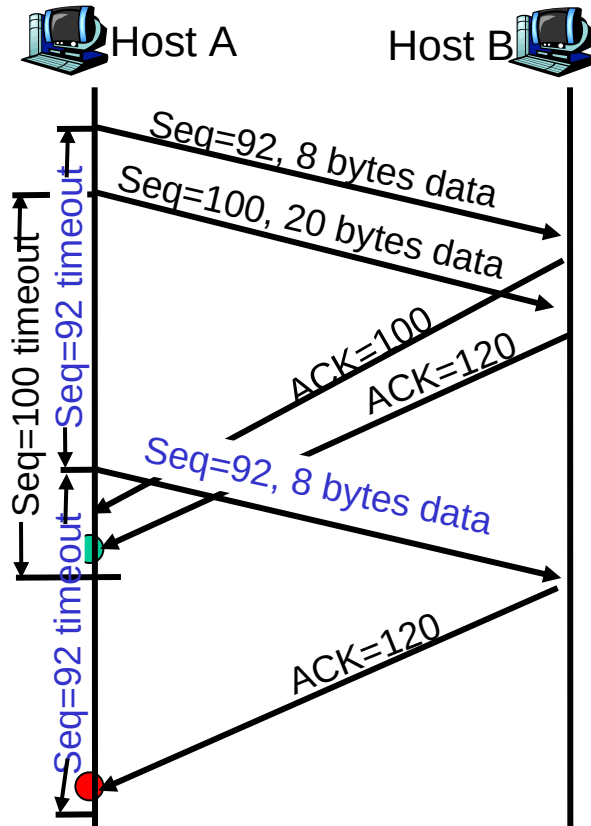


- **“ACK cumulativi” da parte del destinatario**
 - ACK(n) conferma che sono arrivati correttamente i primi n byte dei segmenti dati inviati (Come Go-Back-N)
- **Segmenti arrivati fuori ordine sono salvati nel buffer di ricezione**
 - Uso di buffer in ricezione (Come Ritrasmissione Selettiva)
- Mittente e destinatario gestiscono **due finestre di sequenze** di segmenti da inviare e ricevuti (sliding windows)
- Il destinatario invia un **ACK cumulativo** pari al numero dell'**ultimo byte+1** dell'**ultimo segmento** ricevuto senza errori ed “in sequenza”
- Il mittente **ritrasmette soltanto i segmenti** per i quali non ha ricevuto ACK dal destinatario entro il time-out
 - **Timeout gestito per ogni singolo pacchetto** (come in Ritrasmissione selettiva)

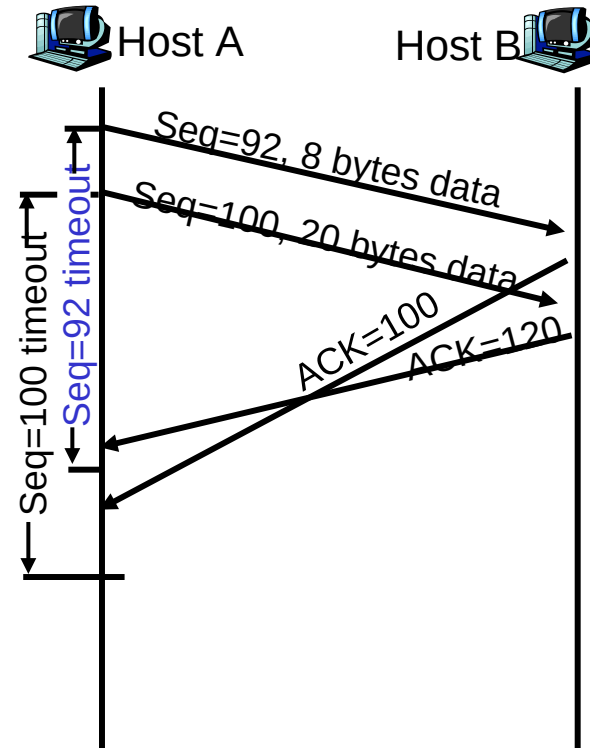
Scenari di ritrasmissione



Scenari di ritrasmissione



L'ack=100 arriva, ma oltre il timeout prestabilito.
L'host A deve rinviare il primo segmento.
Doppia duplicazione: segmento a host B, ack=120 a host A



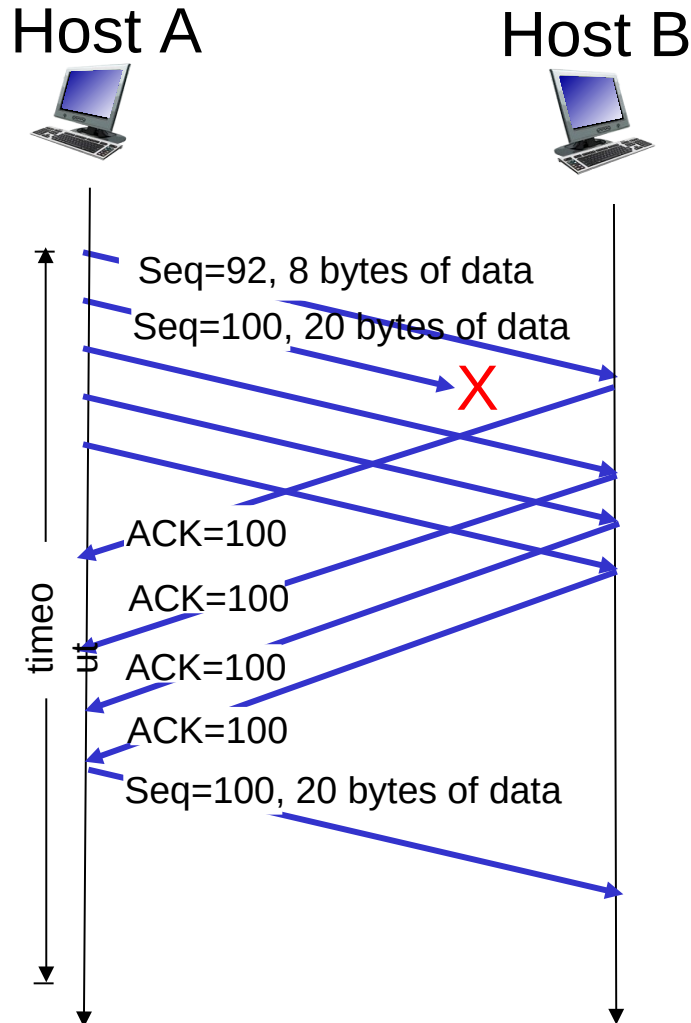
Caso particolare in cui ack=100 non arriva entro il timeout prestabilito, mentre ack=120 arriva addirittura entro il timeout del segmento 92.
NON C'E' RITRASMISSIONE

Ritrasmissione rapida



- Il periodo di time-out è di solito piuttosto lungo
 - Ritardi lunghi prima di ritrasmettere i pacchetti persi → si rischia di dover rispedire anche pacchetti successivi già inviati e ricevuti correttamente (caso precedente)
- **Soluzione:** si identificano i pacchetti persi attraverso la ricezione di ACK duplicati
 - Il mittente spesso manda molti segmenti in un breve lasso di tempo
 - Se un segmento viene perso, ci saranno probabilmente diversi ACK duplicati che arriveranno al server
- Se il mittente riceve **3 ACK per lo stesso segmento**, assume che questo sia stato perso e lo ritrasmette senza aspettarne il timeout (**ritrasmissione rapida**)

Ritrasmissione rapida



Ritrasmissione rapida
Dopo la ricezione di 3
ACK duplicati per uno
Stesso segmento

Osservazione generale



- Nel caso in cui scatti il time-out, la quantità di dati trasmissibile diminuisce perché il **mittente non è in grado di far avanzare la finestra** ed il buffer risulta occupato da altri segmenti
- In generale, più tempo occorre per accorgersi che un segmento è andato perduto, più si **limita la capacità della banda di trasmissione**
 - **necessità di comprendere problemi al più presto**
- **Definizione del timeout è molto importante**

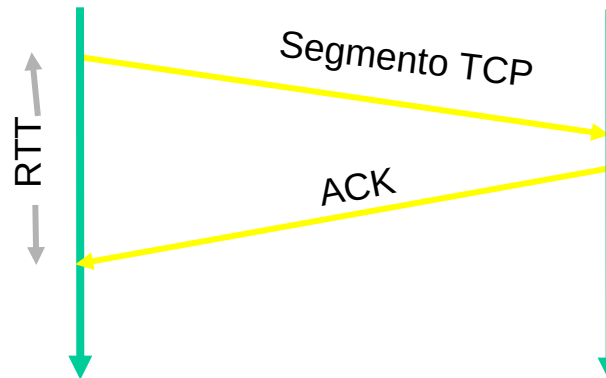
Come stimare il 'time-out'?

Round Trip Time e Timeout



Come stabilire il valore del timeout in trasmissioni TCP?

- **Timeout troppo breve:** si effettuano ritrasmissioni non necessarie
- **Timeout troppo lungo:** reazione lenta alla perdita di segmenti



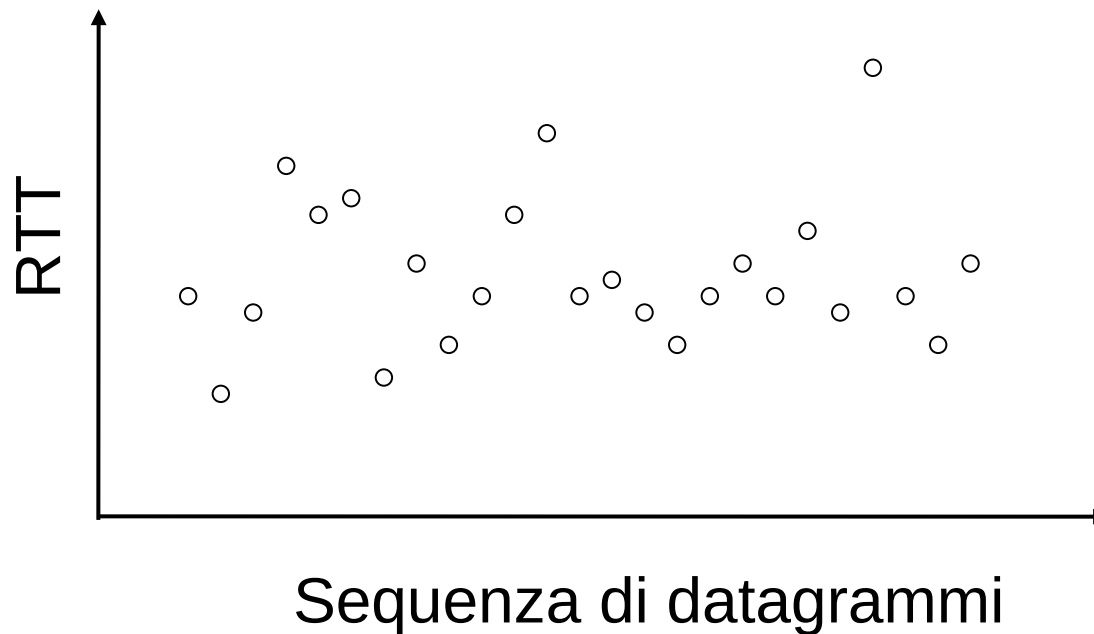
UNICA CERTEZZA: Il timeout deve essere maggiore del Round Trip Time (RTT)

Ma il RTT è stabile? E di quanto deve essere maggiore?

RTT varia continuamente



- **Motivazioni delle differenze del RTT:**
 - **Fluttuazioni delle condizioni di traffico della rete**
 - **Possibili cambiamenti di router nel percorso tra mittente e destinatario**



Scelta del timeout



Inizialmente (obsoleto) si sceglieva:

$$\text{Timeout} = \text{beta} * \text{RTT}_{\text{medio}}$$

dove la raccomandazione era: **beta = 2**

Adesso, si utilizzano alcune stime molto più sofisticate

1) SampleRTT :

- Misura del tempo trascorso dalla trasmissione del segmento alla ricezione del suo ACK
- Ignora ritrasmissioni, segmenti con ack cumulativi

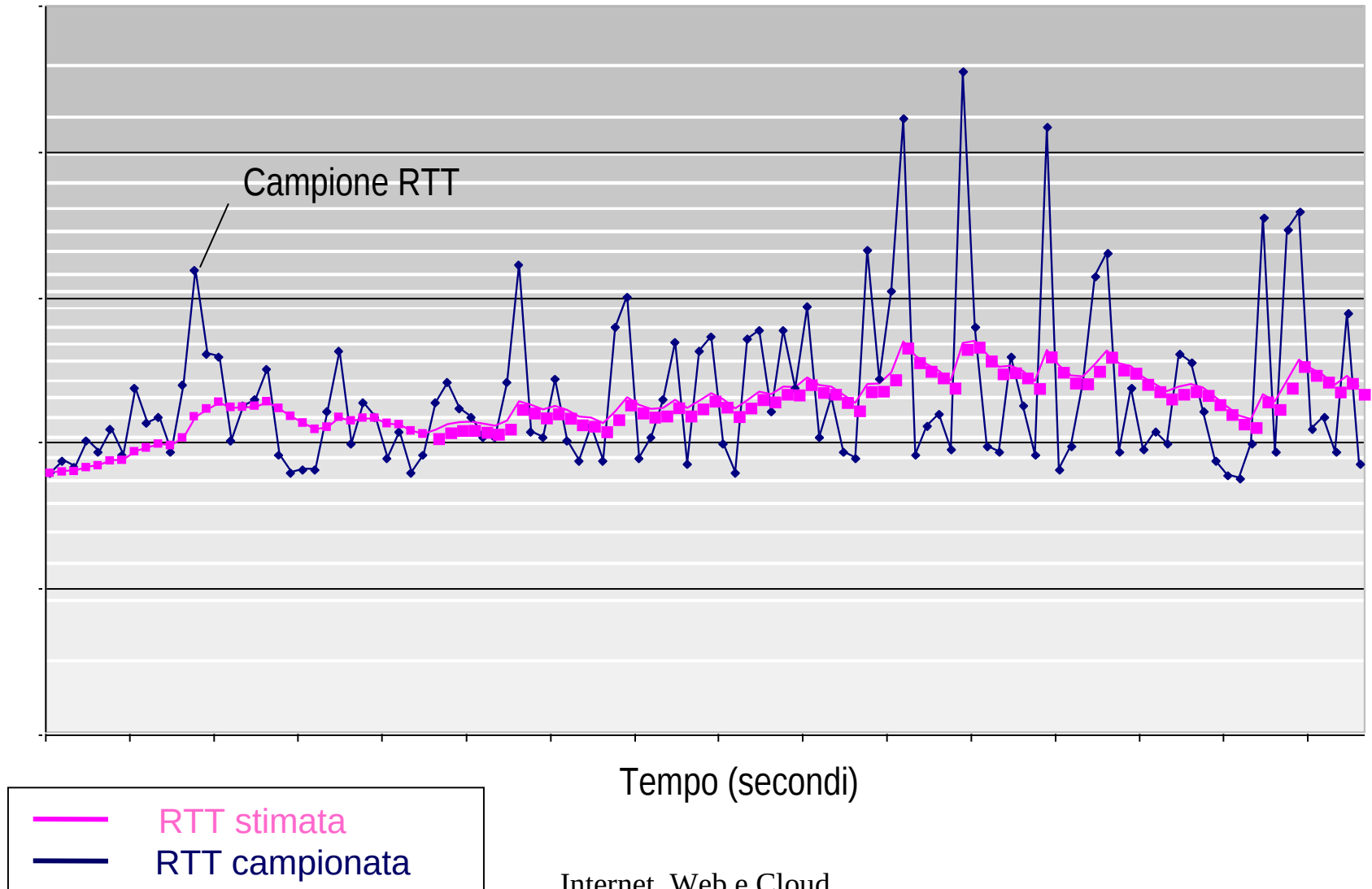
NOTA: il SampleRTT varia dinamicamente, anche significativamente
→ si usa in realtà una media pesata

2) EstimatedRTT:

- Media pesata per stimare RTT al tempo t
- $EstRTT(t) = (1-x)*EstRTT(t-1) + x*SampleRTT(t)$
- Exponential Weighted Moving Average (EWMA)
- L'influenza dei campioni passati diminuisce in modo esponenziale
- Il valore di x è compreso tra 0 e 1 – si sceglie tipicamente il valore:

$x=1/n$ dove n è il numero di campioni di RTT usati per il calcolo
Solitamente $1/8 \rightarrow 0,125$

Esempio stima RTT



Scelta del timeout



Oltre ad avere una **stima del RTT** è importante avere una misura della **sua variabilità** per la scelta del timeout

Variabilità RTT → DevRTT

- **Valore del RTT stimato più un margine di errore**
- $Timeout(t) = EstRTT(t) + 4 * DevRTT(t)$

Dove:

- $DevRTT(t) = (1-b) * DevRTT(t-1) + b * abs[SampRTT(t) - EstRTT(t-1)]$
- *Valore suggerito per $b=0,25$*
- *Nota: se i valori di SampleRTT hanno fluttuazioni limitate → DevRTT sarà piccolo*

"E' vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

E' inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia."